# GUIDA COMPLETA ALL'ESAME: SYSTEMVERILOG & VHDL

> **Guida Operativa e Prontuario per Esami di Sistemi Embedded e Progettazione Digitale**
> *Basata sul riassunto del corso e focalizzata sulla sintesi hardware reale, buone pratiche e pattern d'esame.*

---

## 📌 INTRODUZIONE: LA FORMA MENTIS HARDWARE

Un errore comune durante gli esami di programmazione hardware (HDL) è scrivere codice pensando come in C/Python (esecuzione sequenziale di istruzioni).
Nei linguaggi HDL: 1. **Non si scrive un programma, si descrive un circuito:** ogni riga di codice sintetizzabile genera porte logiche, multiplexer, fili o flip-flop fisici. 2. **Tutto è concorrente (parallelo):** blocchi diversi, assegnamenti continui e moduli operano tutti contemporaneamente. 3. **Il tempo è scandito dal clock:** solo la logica sequenziale (flip-flop) campiona e memorizza lo stato sui fronti di clock ( `posedge clk` ). Tutto il resto è combinatorio e propaga immediatamente (con ritardo di propagazione fisico).

La guida è strutturata in due macro-sezioni: - **PARTE 1: SYSTEMVERILOG (Focus Primario per l'Esame)** - **PARTE 2: VHDL (Sintassi Strutturata e Rigorosa)** - **PARTE 3: TABELLA DI CONVERSIONE RAPIDA E CHECKLIST D'ESAME**

---

# PARTE 1: SYSTEMVERILOG (CORE DELL'ESAME)

---

## 1. STRUTTURA DEL MODULO E TIPI DI DATO

### 1.1 Scheletro del Modulo Parametrizzato

In SystemVerilog moderno (IEEE 1800), le porte e i parametri si dichiarano direttamente nell'intestazione del modulo:

```
`timescale 1ns/1ns

module mio_modulo #(
    parameter int WIDTH = 8,      // Valore di default del parametro
    parameter int DEPTH = 16
)(
    input  logic              clk,
    input  logic              reset,      // Tipicamente asincrono o sincrono attivo alto
    input  logic              enable,
    input  logic [WIDTH-1:0]  dati_in,
    output logic [WIDTH-1:0]  dati_out,
    output logic              flag_pronto
);

    // Dichiarazione segnali interni
    logic [WIDTH-1:0] registro_interno;
    logic             condizione_ok;

    // Logica del modulo ...

endmodule
```

### 1.2 I Tipi di Dato: `logic` vs `wire` vs `bit` (Confronto ed Esempi Pratici)

In SystemVerilog esistono diversi tipi di dato. Capire quando usare ciascuno è fondamentale per evitare errori di compilazione e bug fisici.

**A) `logic` (Lo standard da usare per il 99% dei segnali)**

- **Cosa fa:** Sostituisce i vecchi `reg` e `wire` del Verilog-1995/2001. Può essere guidato sia da un'assegnazione continua `assign` sia dentro blocchi procedurali `always_comb` o `always_ff`.
- **Valori ammessi (Logica a 4 stati):** `'0'`, `'1'`, `'X'` (indeterminato/conflitto), `'Z'` (alta impedenza).
- **Regola fondamentale:** Supporta **UN SOLO DRIVER**. Se per errore due blocchi guidano lo stesso segnale `logic`, il compilatore si blocca immediatamente segnalando errore. Questo protegge da cortocircuiti invisibili.

**B) `wire` (Indispensabile per i Bus Tri-State Condivisi / Multi-Driver)**

- **Quando si usa:** Si usa **esclusivamente** quando più moduli fisici devono pilotare lo **stesso identico filo**, come nei bus dati bidirezionali o tri-state.
- **Perché `logic` qui fallirebbe:** Se provassi a collegare due driver a un segnale `logic`, otterresti un errore del tipo: `ERROR: Variable 'bus' has multiple drivers`. Il tipo `wire`, invece, implementa la **funzione di risoluzione**: se un driver manda `8'h55` e l'altro manda `'bz` (disconnesso), sul `wire` vince `8'h55`. Se entrambi pilotano dati discordi, il `wire` assume valore `'X'` (conflitto/cortocircuito).

```
// ESEMPIO PRATICO: Bus Dati Condiviso tra due Periferiche (Uso corretto di WIRE)
module bus_condiviso_top (
    input  logic       clk,
    input  logic       en_A,       // Se 1, la periferica A guida il bus
    input  logic       en_B,       // Se 1, la periferica B guida il bus
    input  logic [7:0] dati_da_A,
    input  logic [7:0] dati_da_B,
    output logic [7:0] dati_letti
);
    // NOTA BENE: Deve essere 'wire' perché sia A che B guidano questo stesso bus!
    wire [7:0] data_bus;

    // Driver Periferica A
    assign data_bus = en_A ? dati_da_A : 8'hzz;

    // Driver Periferica B
    assign data_bus = en_B ? dati_da_B : 8'hzz;

    // Un lettore sul bus (può tranquillamente essere un segnale logic)
    assign dati_letti = data_bus;
endmodule
```

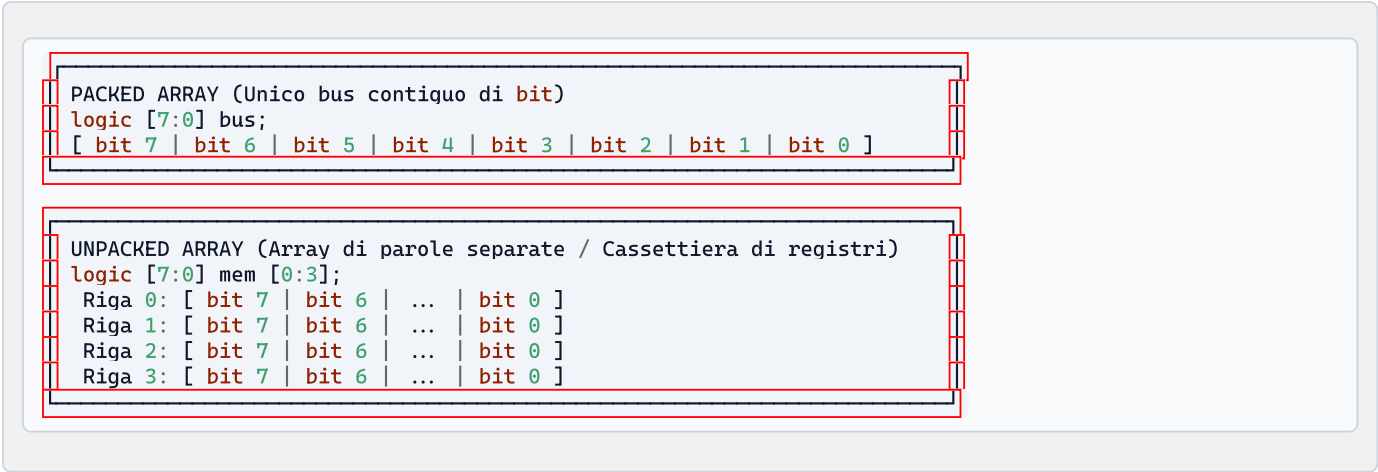
**C) `bit` vs `logic`: Perché NON usare `bit` nel codice sintetizzabile?**

- **La differenza:**
- `logic` è a **4 stati**: `0`, `1`, `X`, `Z`.
- `bit` è a **2 stati**: solo `0` e `1`.
- **Il pericolo d'esame:**
  Nella realtà del silicio (FPGA/ASIC), quando accendi il circuito, i registri non hanno un valore noto: partono in uno stato **sconosciuto ( `X` )** finché non ricevono il reset.
- Se usi `logic`, la simulazione ti mostrerà segnali rossi `X` se hai dimenticato di resettare un flip-flop, permettendoti di correggere il bug!
- Se usi `bit`, il simulatore per definizione inizializza tutto a `0`. Il circuito sembrerà funzionare benissimo in simulazione anche senza reset, ma caricato sulla scheda FPGA reale **non funzionerà mai**, perché il silicio fisico non garantisce lo zero all'accensione!
- **Dove serve `bit` ?** Si usa solo nei **testbench** e nelle simulazioni ad altissima velocità, perché gestire 2 stati invece di 4 richiede molta meno memoria RAM al computer che simula.

---

## 1.3 Packed vs Unpacked Array e Memorie ( `mem` ): Che Vantaggio ho e Quando Usarli?

Questa è una delle distinzioni più importanti e spesso fonte di confusione.




### 1. Sintassi a Confronto

- **Packed Array:** La dimensione è specificata **PRIMA** del nome della variabile. `logic [7:0] dato;` $\to$ Un unico blocco di 8 fili/bit adiacenti.
- **Unpacked Array:** La dimensione è specificata **DOPO** del nome della variabile. `logic [7:0] mem [0:255];` $\to$ Una collezione di 256 elementi distinti, ciascuno dei quali è un vettore packed da 8 bit.

### 2. Che Vantaggio ho dall'usare un Unpacked Array ( `mem` )?

Perché non usare semplicemente un packed array lunghissimo, tipo `logic [2047:0] mem_gigante` ? Ecco i **3 vantaggi concreti e decisivi**:

1. **Inferenza Automatica delle Block RAM (BRAM) su FPGA:**
   Nei chip FPGA (come Xilinx Artix/Kintex o Intel Cyclone), ci sono blocchi fisici di memoria RAM dedicati ad alte prestazioni (Block RAM).
2. Se scrivi un **unpacked array** indicizzato con clock e write-enable, il tool di sintesi (Vivado o Quartus) riconosce il pattern ed **inserisce direttamente una BRAM hardware dedicata**, senza consumare logica dell'FPGA.
3. Se invece usassi un packed array gigante, il sintetizzatore cercherebbe di costruire migliaia di flip-flop discreti collegati a una mostruosa rete di multiplexer a porte logiche (LUT), esaurendo le risorse del chip e rendendo il circuito lentissimo.
4. **Modellazione Naturale di Banchi di Registri (Register File CPU):**
   Nei processori (es. RISC-V a 32 registri da 32 bit), l'unpacked array permette di leggere contemporaneamente più registri usando indirizzi distinti: `regfile[rs1]` e `regfile[rs2]` .
5. **Indicizzazione Pulita e Sicura:**
   Scrivere `mem[indirizzo]` accede direttamente all'intera parola di memoria, esattamente come faresti in un array C, senza dover calcolare manualmente gli offset di bit moltiplicati ( `mem_gigante[addr*8 +: 8]` ).

### 3. Cosa si può e cosa NON si può fare

| Operazione | Packed Array ( `logic [7:0] a` ) | Unpacked Array ( `logic [7:0] mem [0:255]` ) |
|---|---|---|
| Operazioni Aritmetiche ( `+` , `-` , `*` ) | ✅ Sì ( `a + 1` ) | ❌ No (non puoi fare `mem + 1` ) |
| Scorrimenti e Shift ( `<<` , `>>` ) | ✅ Sì ( `a << 2` ) | ❌ No |
| Operazioni Logiche Bitwise ( `&` , `\|` , `^` ) | ✅ Sì ( `a & b` ) | ❌ No su tutto l'array (Sì sul singolo elemento `mem[i] & b` ) |
| Assegnamento da intero | ✅ Sì ( `a = 8'd42;` ) | ❌ No (devi assegnare `mem[i] = 8'd42;` ) |
| Accesso a singolo elemento | Accede a singolo bit `a[3]` | Accede all'intera parola `mem[3]` |

### Esempio Pratico Completo: Register File di una CPU (32 Registri x 32 Bit)

Questo è il classico blocco che richiede l'uso dell'unpacked array:

```
module register_file (
    input  logic        clk,
    input  logic        write_en,
    input  logic [4:0]  read_addr_1,  // 5 bit per indirizzare 32 registri (2^5 = 32)
    input  logic [4:0]  read_addr_2,
    input  logic [4:0]  write_addr,
    input  logic [31:0] write_data,
    output logic [31:0] read_data_1,
    output logic [31:0] read_data_2
);
    // UNPACKED ARRAY: 32 elementi, ciascuno è una parola PACKED di 32 bit
    logic [31:0] rf [0:31];

    // Scrittura SINCRONA sul fronte di clock
    always_ff @(posedge clk) begin
        if (write_en) begin
            // In molte CPU (es. RISC-V), il registro 0 è sempre cablato a 0
            if (write_addr ≠ 5'd0)
                rf[write_addr] ≤ write_data;
        end
    end

    // Lettura ASINCRONA combinatoria (due porte indipendenti)
    assign read_data_1 = (read_addr_1 == 5'd0) ? 32'd0 : rf[read_addr_1];
    assign read_data_2 = (read_addr_2 == 5'd0) ? 32'd0 : rf[read_addr_2];

endmodule
```

---

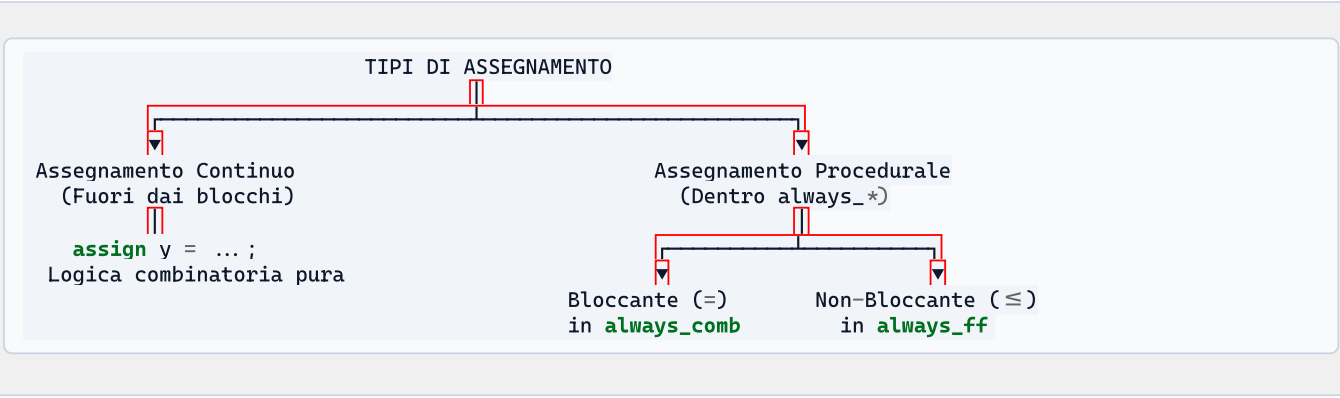
### 1.4 Notazione dei Numeri e Operazioni sui Vettori

- Sintassi letterale: `<N_bit>'<base><valore>`
- `4'b1010` (4 bit binari)
- `8'hFF` (8 bit esadecimali = 255)
- `16'd42` (16 bit decimali)
- `'0` o `'1` : estende automaticamente lo zero o l'uno su tutta la larghezza del bus, qualunque essa sia! (Ottimo per codice parametrizzato: `q ≤ '0;` ).
- **Concatenazione e Replicazione:**
- Concatenazione: `{a, b, c}` unisce i bit di $a$, $b$ e $c$.
- Replicazione: `{4{a}}` replica il bit/bus $a$ per 4 volte.
- Esempio: `{c[2:1], {3{d[0]}}, 3'b101}` unisce 2 bit di $c$, 3 copie di $d[0]$ e 3 bit letterali.
- **Signed vs Unsigned (Complemento a Due):**
- Di default i vettori sono unsigned.
- Per fare algebra con segno: `logic signed [WIDTH-1:0] a, b;`
- Oppure cast dinamico: `$signed(a) + $signed(b);`
- **Attenzione all'estensione del segno:** se estendi un numero unsigned con zeri in testa basta `{ {4{1'b0}}, a }`. Se il numero è con segno (complemento a due), devi replicare il MSB (bit di segno): `{ {4{a[WIDTH-1]}}, a }`.

---

## 2. I TIPI DI ASSEGNAMENTO E LE REGOLE D'ORO

Questa è la parte dove gli studenti perdono più punti negli esami. Imparare a memoria e comprendere queste regole.




### 2.1 Assegnamento Continuo ( `assign` )

- **Dove:** Fuori da qualsiasi blocco `always`.
- **Hardware:** Modella un filo o una rete combinatoria immediata. Appena cambia un ingresso a destra, l'uscita a sinistra cambia istantaneamente.
- **Esempio:** `systemverilog assign out = in1 & in2; assign mux_out = sel ? d1 : d0; // Operatore ternario MUX`

### 2.2 Assegnamento Procedurale Bloccante ( `=` )

- **Dove: SOLO** dentro blocchi combinatori `always_comb` (o funzioni).
- **Semantica:** Viene eseguito immediatamente, riga per riga, come in un programma sequenziale. Le righe successive vedono **subito** il nuovo valore calcolato dalla riga precedente.
- **Perché serve:** Per calcoli intermedi a cascata (es. calcolo del riporto di una somma, poi usato subito nella riga successiva per la somma finale).

### 2.3 Assegnamento Procedurale Non-Bloccante ( `≤` )

- **Dove: SOLO** dentro blocchi sequenziali sincroni `always_ff @(posedge clk)`.
- **Semantica:** Al fronte di clock, tutti i valori a destra dell'uguale vengono **campionati contemporaneamente**. L'aggiornamento dei registri a sinistra avviene solo alla fine dell'istante di clock.
- **Perché è vitale:** Modella i **veri Flip-Flop fisici** in parallelo.

---

### 2.4 LE 5 REGOLE D'ORO PER L'ESAME (NON VIOLARE MAI)

1. **Nei blocchi sequenziali ( `always_ff` ): USA ESCLUSIVAMENTE `≤` (non-blocking).**
2. **Nei blocchi combinatori procedurali ( `always_comb` ): USA ESCLUSIVAMENTE `=` (blocking).**
3. **NON mischiare MAI `=` e `≤` nello stesso blocco `always`.**
4. **REGOLA DEL SINGOLO DRIVER:** Un segnale non può essere assegnato in due blocchi `always` diversi o sia da un `assign` che da un `always`. Ogni filo ha un solo generatore.
5. **PREVENZIONE DEI LATCH:** In un blocco `always_comb`, assegna sempre un valore a tutte le uscite per tutte le combinazioni possibili di ingresso (oppure metti un default all'inizio).

---

### 2.5 DIMOSTRAZIONI PRATICHE: COSA SUCCEDE SE SBAGLI I TIPI DI ASSEGNAMENTO?

Gli esaminatori adorano fare domande su cosa accade quando si sbaglia operatore. Ecco le 3 dimostrazioni pratiche:

#### 💥 Caso 1: Cosa succede se usi `=` (bloccante) in un blocco sequenziale?

Immagina di voler costruire una pipeline o un registro a scorrimento (shift register) a 3 stadi:

```
// CODICE ERRATO CON ASSEGNAMENTO BLOCCANTE (=)
always_ff @(posedge clk) begin
    r1 = in_data;
    r2 = r1;
    r3 = r2;
end
```

**Cosa fa fisicamente il circuito? (Disastro!)**
Il sintetizzatore valuta le righe in sequenza immediata: 1. `r1` prende `in_data`. 2. Nella riga successiva, `r2` legge il NUOVO valore di `r1` appena scritto! Quindi `r2 = in_data`. 3. Nella terza riga, `r3` legge il NUOVO valore di `r2`! Quindi `r3 = in_data`. - **Risultato hardware:** I registri intermedi collassano! Il dato attraversa tutti e tre gli stadi nello stesso ciclo di clock. Invece di 3 Flip-Flop in cascata, il sintetizzatore sintetizza **UN SOLO FLIP-FLOP**. La pipeline è distrutta.

```
// CODICE CORRETTO CON ASSEGNAMENTO NON-BLOCCANTE (≤)
always_ff @(posedge clk) begin
    r1 ≤ in_data;
    r2 ≤ r1;
    r3 ≤ r2;
end
```

**Cosa fa fisicamente questo codice? (Corretto!)**
Al fronte di clock, tutti i valori destri ( `in_data`, il vecchio `r1`, il vecchio `r2` ) vengono campionati **simultaneamente**. Solo dopo, le uscite vengono aggiornate. Si ottiene esattamente un ritardo di 3 cicli di clock con 3 Flip-Flop fisici indipendenti.

---

### 💥 Caso 2: Cosa succede se usi `≤` (non-bloccante) in un blocco combinatorio?

Considera un blocco combinatorio che calcola un risultato intermedio per poi usarlo subito:

```
// CODICE ERRATO CON NON-BLOCCANTE IN LOGICA COMBINATORIA
always_comb begin
    temp ≤ a + b;
    risultato ≤ temp * 2; // ERRORE!
end
```

**Cosa accade in simulazione?**
Poiché `≤` pianifica l'aggiornamento per la fine del ciclo temporale (delta delay), la seconda riga userà il **VECCHIO valore di `temp`**, non quello appena calcolato dalla somma `a + b`!
L'uscita `risultato` mostrerà un dato errato o genererà oscillazioni/glitch nella simulazione.
**Correzione:** In `always_comb` usa sempre `=`, così `temp` è calcolato e disponibile istantaneamente per le righe successive:

```
always_comb begin
    temp = a + b;
    risultato = temp * 2; // CORRETTO: usa il valore appena calcolato
end
```

---

### 💥 Caso 3: L'errore del Doppio Driver (Multi-Driver Violation)

Questo è un errore classico commesso dagli studenti quando dividono la logica:

```
// ERRORE GRAVISSIMO D'ESAME: Due blocchi che pilotano la stessa variabile
always_ff @(posedge clk) begin
    if (condizione_1)
        led ≤ dati_sensore;
end

always_ff @(posedge clk) begin
    if (condizione_2)
        led ≤ dati_allarme; // ERRORE DI SINTESI: 'led' has multiple procedural drivers!
end
```

**Spiegazione:** In silicio non puoi collegare le uscite di due flip-flop diversi allo stesso filo `led` senza un multiplexer, altrimenti causeresti un cortocircuito se entrambi si accendono contemporaneamente.
**Soluzione:** Riunire tutta la logica di pilotaggio in **un unico blocco sequenziale**:

```
// SOLUZIONE CORRETTA: Singolo blocco con priorità chiara
always_ff @(posedge clk) begin
    if (condizione_2)
        led ≤ dati_allarme; // Priorità all'allarme
    else if (condizione_1)
        led ≤ dati_sensore;
    else
        led ≤ '0;
end
```

---

## 3. TUTTE LE PORTE LOGICHE E BLOCCHI COMBINATORI (IMPLEMENTAZIONE)

### 3.1 Porte Logiche Elementari (Bitwise)

Tutte le porte logiche possono essere implementate con `assign` continuo o dentro `always_comb`:

| Porta | Operatore SV | Esempio Continuo | Funzione Booleana |
|---|---|---|---|
| **NOT** | `~` | `assign y = ~a;` | $Y = \overline{A}$ |
| **AND** | `&` | `assign y = a & b;` | $Y = A \cdot B$ |
| **OR** | `\|` | `assign y = a \| b;` | $Y = A + B$ |
| **XOR** | `^` | `assign y = a ^ b;` | $Y = A \oplus B$ |
| **NAND** | `~&` o `~(a & b)` | `assign y = ~(a & b);` | $Y = \overline{A \cdot B}$ |
| **NOR** | `~\|` o `~(a \| b)` | `assign y = ~(a \| b);` | $Y = \overline{A + B}$ |
| **XNOR** | `~^` o `^~` | `assign y = ~(a ^ b);` | $Y = \overline{A \oplus B}$ |

**Operatori di Riduzione (Unari):**
Applicano l'operatore a tutti i bit di un singolo vettore: - `&bus` -> Restituisce 1 se TUTTI i bit di `bus` sono 1 (grande porta AND a N ingressi). - `|bus` -> Restituisce 1 se ALMENO un bit di `bus` è 1 (grande porta OR). - `^bus` -> Calcola la parità (XOR a cascata di tutti i bit).

---

## 3.2 Multiplexer (MUX)

### MUX 2:1 (Livello Continuo con operatore ternario)

```
module mux2 #(parameter WIDTH = 8)(
    input  logic [WIDTH-1:0] d0, d1,
    input  logic             s,
    output logic [WIDTH-1:0] y
);
    assign y = s ? d1 : d0;
endmodule
```

### MUX 4:1 (Con `case` e `always_comb`)

```
module mux4 #(parameter WIDTH = 8)(
    input  logic [WIDTH-1:0] d0, d1, d2, d3,
    input  logic [1:0]       s,
    output logic [WIDTH-1:0] y
);
    always_comb begin
        case (s)
            2'b00:   y = d0;
            2'b01:   y = d1;
            2'b10:   y = d2;
            2'b11:   y = d3;
            default: y = '0; // Buona pratica sempre
        endcase
    end
endmodule
```

---

## 3.3 Decodificatori (Decoders) e Priority Encoders

### Decodificatore 3:8 con `case`

```
module decoder3_8 (
    input  logic [2:0] a,
    output logic [7:0] y
);
    always_comb begin
        case (a)
            3'b000:  y = 8'b00000001;
            3'b001:  y = 8'b00000010;
            3'b010:  y = 8'b00000100;
            3'b011:  y = 8'b00001000;
            3'b100:  y = 8'b00010000;
            3'b101:  y = 8'b00100000;
            3'b110:  y = 8'b01000000;
            3'b111:  y = 8'b10000000;
            default: y = 8'b00000000;
        endcase
    end
endmodule
```

### Decodificatore Parametrico $N \to 2^N$ (Tecnica d'esame elegante)

Invece di scrivere centinaia di righe di `case`:

```
module decoder_param #(parameter N = 3)(
    input  logic [N-1:0]         a,
    output logic [(1 << N)-1:0]  y
);
    always_comb begin
        y = '0;       // 1. Azzera tutte le 2^N uscite
        y[a] = 1'b1;  // 2. Accende SOLO l'uscita selezionata dall'indirizzo a
    end
endmodule
```

**Priority Encoder con `casez` (Don't-Care `?`)**

Un encoder con priorità assegna l'uscita al bit più significativo attivo:

```
module priority_encoder_4 (
    input  logic [3:0] a,
    output logic [1:0] y,
    output logic       valido
);
    always_comb begin
        valido = 1'b1;
        casez (a)
            4'b1???: y = 2'b11; // Se a[3]=1, il resto non conta
            4'b01??: y = 2'b10; // Se a[3]=0 e a[2]=1
            4'b001?: y = 2'b01;
            4'b0001: y = 2'b00;
            default: begin
                y = 2'b00;
                valido = 1'b0; // Nessun ingresso attivo
            end
        endcase
    end
endmodule
```

---

## 3.4 Buffer Tri-State e Bus Condivisi ( `'bz` )

I buffer tri-state permettono a più componenti di condividere lo stesso bus di comunicazione. Quando non abilitati, pongono l'uscita in **alta impedenza** ( `'bz` ), scollegandosi fisicamente.

```
module tristate_bus_driver #(parameter WIDTH = 8)(
    input  logic [WIDTH-1:0] dati_in,
    input  logic             enable, // Attivo alto
    inout  wire  [WIDTH-1:0] bus     // NOTA: 'wire' è obbligatorio per bus multi-driver!
);
    assign bus = enable ? dati_in : 'bz;
endmodule
```

---

### 3.5 Aritmetica: Full Adder e Sommatore con Rilevamento Overflow

#### Full Adder (Bitwise)

```
module full_adder (
    input  logic a, b, cin,
    output logic s, cout
);
    logic p, g;
    assign p = a ^ b;       // Propagate
    assign g = a & b;       // Generate
    assign s = p ^ cin;
    assign cout = g | (p & cin);
endmodule
```

#### Sommatore N-bit con Overflow in Complemento a Due

> ⚠️ **IMPORTANT:** **Come si calcola l'overflow in complemento a due?**
> L'overflow si verifica se e solo se la somma di due numeri con lo stesso segno produce un risultato con segno opposto!
> Formula: `ovf = (a[N-1] == b[N-1]) && (sum[N-1] != a[N-1])`

```
module adder_subtractor #(parameter N = 8)(
    input  logic signed [N-1:0] a,
    input  logic signed [N-1:0] b,
    input  logic                sub, // 0 = somma, 1 = sottrazione
    output logic signed [N-1:0] res,
    output logic                overflow
);
    logic signed [N-1:0] b_effettivo;

    // Se sottrazione, inverti b per la regola a - b = a + (~b + 1)
    assign b_effettivo = sub ? -b : b;
    assign res = a + b_effettivo;

    // Controllo overflow
    assign overflow = (a[N-1] == b_effettivo[N-1]) && (res[N-1] ≠ a[N-1]);
endmodule
```

---

## 4. LOGICA SEQUENZIALE: FLIP-FLOP, REGISTRI E CONTATORI

La logica sequenziale richiede **memoria**. I blocchi sequenziali si scrivono sempre con `always_ff @(posedge clk)`.

### 4.1 Flip-Flop D: Reset Sincrono vs Asincrono

#### Reset Asincrono (Attivo Alto)

Il reset ha effetto **immediato**, indipendentemente dal clock. Va inserito nella sensitivity list:

```
always_ff @(posedge clk or posedge reset) begin
    if (reset)
        q ≤ '0;
    else
        q ≤ d;
end
```

**Reset Sincrono (Attivo Alto)**

Il reset viene valutato **solo al fronte di salita del clock**:

```
always_ff @(posedge clk) begin
    if (reset)
        q ≤ '0;
    else
        q ≤ d;
end
```

---

## 4.2 Registro con Reset ed Enable (Il blocco d'esame per eccellenza)

Un registro con segnale di abilitazione `en`: se `en=0`, mantiene il proprio valore senza cambiare.

```
module register_en #(parameter N = 8)(
    input  logic         clk,
    input  logic         reset,
    input  logic         en,
    input  logic [N-1:0] d,
    output logic [N-1:0] q
);
    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            q ≤ '0;
        else if (en)
            q ≤ d;
        // else implicito: q mantiene il suo valore (non genera latch perché siamo su fronte di clock!)
    end
endmodule
```

---

## 4.3 Registri a Scorrimento (Shift Register)

Uno Shift Register a 4 bit con caricamento parallelo (`load`) e scorrimento a sinistra:

```
module shift_reg (
    input  logic       clk,
    input  logic       reset,
    input  logic       load,
    input  logic       sin,   // Serial in
    input  logic [3:0] din,   // Parallel in
    output logic [3:0] q,     // Parallel out
    output logic       sout   // Serial out
);
    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            q ≤ 4'b0000;
        else if (load)
            q ≤ din;
        else
            q ≤ {q[2:0], sin}; // Scorrimento: shift a sinistra inserendo sin
    end

    assign sout = q[3]; // L'uscita seriale è il bit MSB che "cade fuori"
endmodule
```

---

### 4.4 Contatore Up/Down Parametrizzato

```
module counter_up_down #(parameter N = 8)(
    input  logic         clk,
    input  logic         reset,
    input  logic         en,
    input  logic         up_down, // 1 = conta su, 0 = conta giù
    output logic [N-1:0] count
);
    always_ff @(posedge clk or posedge reset) begin
        if (reset)
            count ≤ '0;
        else if (en) begin
            if (up_down)
                count ≤ count + 1'b1;
            else
                count ≤ count - 1'b1;
        end
    end
endmodule
```

---

### 4.5 Sincronizzatore a 2 Flip-Flop (Antidoto alla Metastabilità)

Quando un segnale asincrono dall'esterno (es. un pulsante) entra nel circuito, può violare i tempi di setup/hold del primo flip-flop. Per sincronizzarlo si usano due FF in cascata:

```
module synchronizer (
    input  logic clk,
    input  logic async_in,
    output logic sync_out
);
    logic n1;

    always_ff @(posedge clk) begin
        n1       ≤ async_in; // Primo FF (potrebbe diventare metastabile)
        sync_out ≤ n1;       // Secondo FF (filtra la metastabilità)
    end
endmodule
```

---

### 4.6 IL PERICOLO DEI LATCH: COME RICONOSCERLI ED EVITARLI

> 🚨 **WARNING:** **Cos'è un Latch?**
> Un latch è un elemento di memoria sensibile al **livello** (trasparente quando il clock è alto/basso) e NON al fronte.
> Nei circuiti sincroni digitali moderni per FPGA e ASIC, **I LATCH INDESIDERATI SONO UN ERRORE GRAVISSIMO** perché introducono glitch e rendono impossibile la static timing analysis (STA).

**Da cosa nasce un latch indesiderato?**

Nasce quando un blocco combinatorio ( `always_comb` ) **non assegna un valore a un segnale in tutti i possibili rami logici** (ad esempio un `if` senza `else` o un `case` senza `default` ). Il sintetizzatore conclude: "*In quel caso il valore deve essere mantenuto, quindi devo creare una memoria fisica (latch)!*"

❌ **CODICE ERRATO (Genera Latch):**

```
always_comb begin
    if (a == 2'b01)
        y = 1'b1;
    // ERRORE: cosa succede se a vale 00, 10 o 11? y deve memorizzare il passato → LATCH!
end
```

✅ **CODICE CORRETTO (Metodo del Default Assignment):**

```
always_comb begin
    y = 1'b0; // VALORE DI DEFAULT all'inizio del blocco!
    if (a == 2'b01)
        y = 1'b1;
    // Adesso per qualunque valore diverso da 01, y vale 0. Nessun latch inferito!
end
```

---

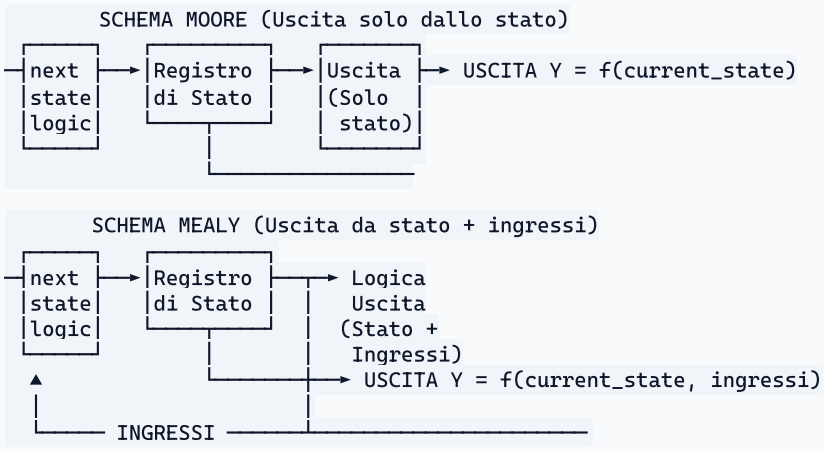
## 5. MACCHINE A STATI FINITI (FSM): MOORE E MEALY

Le FSM sono l'argomento centrale di quasi tutti gli esami pratici. Una FSM combina registri (memoria) e porte logiche per modellare sistemi con un numero finito di condizioni operative.

---

### 5.1 Confronto Strutturale: Moore vs Mealy (È davvero complicata Mealy?)

**La risposta è NO! Una macchina di Mealy NON è affatto più complicata.** La struttura del codice è identica al 90% a quella di Moore.
L'unica vera differenza risiede **in come viene calcolata l'uscita**:



### Tabella Comparativa per l'Esame

| Caratteristica | Macchina di MOORE | Macchina di MEALY |
|---|---|---|
| **Formula dell'Uscita** | $Y = f(\text{current_state})$ | $Y = f(\text{current_state}, \text{ingressi})$ |
| **Dove si trova l'uscita nel grafo** | **Dentro il cerchio** dello stato ($S_i / Y$) | **Sull'arco (freccia)** di transizione ($A / Y$) |
| **Reattività** | Ritardata di 1 ciclo di clock (l'uscita cambia solo dopo essere entrati nel nuovo stato) | **Immediata** (reagisce nello stesso ciclo di clock appena cambia l'ingresso) |
| **Numero di Stati** | Spesso richiede **più stati** | Spesso richiede **1 stato in meno** (risparmia Flip-Flop!) |
| **Glitch / Rumore** | Uscite stabilissime, immuni da glitch | Possibili glitch se gli ingressi oscillano (perché l'uscita è collegata direttamente all'ingresso via porte logiche) |

---

## 5.2 Lo Schema Standard a 3 Blocchi d'Esame

Sia per Moore che per Mealy, l'architettura del codice raccomandata è sempre la stessa: 1. **Blocco 1 (Sequenziale):** Registro di stato che memorizza `current_state ≤ next_state;` sul fronte di clock. 2. **Blocco 2 (Combinatorio):** Calcolo di `next_state` tramite `case (current_state)`. 3. **Blocco 3 (Uscita):** - **In Moore:** Guarda solo `current_state`. - **In Mealy:** Guarda `current_state` E gli `ingressi`.

---

## 5.3 Esempio a Confronto Diretto: Riconoscitore di Sequenza "101"

Vogliamo riconoscere la sequenza seriale `101` in arrivo sul pin `a`, con sovrapposizione (overlapping, es. `10101` genera due impulsi).

### 🟢 Implementazione MOORE (Richiede 4 Stati)

In Moore dobbiamo creare uno stato finale separato ($S_3$) che significa: "*Siamo nello stato in cui la sequenza è stata riconosciuta*".

```
module seq_detector_moore (
    input  logic clk,
    input  logic reset,
    input  logic a,
    output logic y
);
    // 4 stati → servono 2 bit di registro (2^2 = 4)
    typedef enum logic [1:0] {
        S0, // Reset / nessun bit valido
        S1, // Visto '1'
        S2, // Visto '10'
        S3  // Visto '101' → Sequenza riconosciuta!
    } state_t;

    state_t current_state, next_state;

    // BLOCCO 1: Registro di Stato
    always_ff @(posedge clk or posedge reset) begin
        if (reset) current_state ≤ S0;
        else       current_state ≤ next_state;
    end

    // BLOCCO 2: Calcolo Prossimo Stato
    always_comb begin
        next_state = current_state; // Default anti-latch
        case (current_state)
            S0: next_state = a ? S1 : S0;
            S1: next_state = a ? S1 : S2;
            S2: next_state = a ? S3 : S0; // Se arriva 1 andiamo nello stato di successo S3
            S3: next_state = a ? S1 : S2; // Overlapping: l'ultimo '1' può essere l'inizio di un nuovo '101'
            default: next_state = S0;
        endcase
    end

    // BLOCCO 3: Uscita MOORE (Dipende SOLO dallo stato presente!)
    assign y = (current_state == S3);

endmodule
```

---

### 🔵 Implementazione MEALY (Richiede solo 3 Stati!)

In Mealy **non serve lo stato $S_3$**! Quando siamo nello stato $S_2$ (abbiamo già visto `10`) e arriva l'ingresso $a=1$, la sequenza è completa adesso! L'uscita si accende immediatamente sull'arco di transizione verso $S_1$.

```
module seq_detector_mealy (
    input  logic clk,
    input  logic reset,
    input  logic a,
    output logic y
);
    // Solo 3 stati! Risparmiamo logica!
    typedef enum logic [1:0] {
        S0, // Reset / nessun bit
        S1, // Visto '1'
        S2  // Visto '10'
    } state_t;

    state_t current_state, next_state;

    // BLOCCO 1: Registro di Stato (IDENTICO A MOORE!)
    always_ff @(posedge clk or posedge reset) begin
        if (reset) current_state ≤ S0;
        else       current_state ≤ next_state;
    end

    // BLOCCO 2: Calcolo Prossimo Stato
    always_comb begin
        next_state = current_state; // Default anti-latch
        case (current_state)
            S0: next_state = a ? S1 : S0;
            S1: next_state = a ? S1 : S2;
            S2: next_state = a ? S1 : S0; // Torna a S1 se a=1 (overlapping), altrimenti a S0
            default: next_state = S0;
        endcase
    end

    // BLOCCO 3: Uscita MEALY (Dipende da current_state E dall'ingresso 'a'!)
    // L'uscita va a 1 SE siamo nello stato S2 E l'ingresso attuale 'a' vale 1!
    assign y = (current_state == S2 && a == 1'b1);

endmodule
```

> 🎯 **Guarda la differenza tra i due blocchi 3:**
> - In Moore: `assign y = (current_state == S3);`
> - In Mealy: `assign y = (current_state == S2 && a == 1'b1);`
> Tutto il resto del codice è praticamente identico!

---

### 5.4 Esempio Reale dalle Slide del Corso (`riassunto_vhdl.pdf`, pag. 69-71)

Nelle slide del corso viene presentato un classico esercizio d'esame:

> **Consegna:** Progettare una FSM con un ingresso seriale `a` e un'uscita `y`.
> L'uscita `y` deve valere `1` quando l'ingresso attuale `a` è uguale a quello che era **nei due cicli di clock precedenti** (ovvero la sequenza è `0?0` con $a=0$, oppure `1?1` con $a=1$).

Perché le slide specificano che è una **Mealy FSM**? 1. Serve **memoria (stato)** per ricordare la storia dei bit passati. 2. Serve reagire **subito all'ingresso attuale** `a` per confrontarlo coi bit precedenti nello stesso ciclo di clock. 3. L'uscita dipende sia dallo stato che dall'ingresso: $$\text{assign } y = ((state == S1 \mid state == S2) \ \& \ \sim a) \ \mid \ ((state == S3 \mid state == S4) \ \& \ a);$$

Ecco il codice completo da slide riscritto e commentato:

```
module pattern_recognizer_mealy (
    input  logic clk,
    input  logic reset,
    input  logic a,
    output logic y
);
    // Stati simbolici per ricordare la storia dei bit precedenti
    typedef enum logic [2:0] {
        S0, // Iniziale / Reset
        S1, // Visto 0
        S2, // Visto 00
        S3, // Visto 1
        S4  // Visto 11
    } state_t;

    state_t state, nextstate;

    // 1. Registro di stato
    always_ff @(posedge clk or posedge reset) begin
        if (reset) state ≤ S0;
        else       state ≤ nextstate;
    end

    // 2. Logica di transizione (Next State)
    always_comb begin
        case (state)
            S0: if (a) nextstate = S3; else nextstate = S1;
            S1: if (a) nextstate = S3; else nextstate = S2;
            S2: if (a) nextstate = S4; else nextstate = S2;
            S3: if (a) nextstate = S4; else nextstate = S1;
            S4: if (a) nextstate = S4; else nextstate = S2;
            default: nextstate = S0;
        endcase
    end

    // 3. Logica di uscita MEALY
    // y vale 1 se la storia era un bit 0 (stati S1 o S2) ed entra un nuovo 0 (~a),
    // OPPURE se la storia era un bit 1 (stati S3 o S4) ed entra un nuovo 1 (a).
    assign y = ((state == S1 || state == S2) && ~a) ||
               ((state == S3 || state == S4) &&  a);

endmodule
```

---

### 5.5 Esempio: Divisore di Frequenza per 3 (Moore Puro)

Dalla slide del corso: una macchina autonoma che genera un impulso ogni 3 fronti di clock (senza ingressi esterni a parte clk e reset).

```
module divide_by_3 (
    input  logic clk,
    input  logic reset,
    output logic y
);
    typedef enum logic [1:0] { S0, S1, S2 } state_t;
    state_t current_state, next_state;

    // 1. Registro di stato
    always_ff @(posedge clk or posedge reset) begin
        if (reset) current_state ≤ S0;
        else       current_state ≤ next_state;
    end

    // 2. Prossimo stato a rotazione ciclica
    always_comb begin
        case (current_state)
            S0:      next_state = S1;
            S1:      next_state = S2;
            S2:      next_state = S0;
            default: next_state = S0;
        endcase
    end

    // 3. Output di Moore: y vale 1 SOLO nello stato S0
    assign y = (current_state == S0);
endmodule
```

---

## 6. MEMORIE (RAM E ROM)

### 6.1 RAM Sincrona Single-Port con Enable di Scrittura ( `we` )

```
module ram_sp #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 6 // 2^6 = 64 posizioni
)(
    input  logic                  clk,
    input  logic                  we,   // Write Enable
    input  logic [ADDR_WIDTH-1:0] addr,
    input  logic [DATA_WIDTH-1:0] din,
    output logic [DATA_WIDTH-1:0] dout
);
    // Array di memoria: unpacked array
    logic [DATA_WIDTH-1:0] mem [0:(1 << ADDR_WIDTH)-1];

    // Scrittura sincrona
    always_ff @(posedge clk) begin
        if (we)
            mem[addr] ≤ din;
    end

    // Lettura asincrona combinatoria (oppure sincrona registrando dout)
    assign dout = mem[addr];
endmodule
```

---

## 7. COME SCRIVERE UN TESTBENCH EFFICACE (PER SIMULAZIONE)

Per verificare i progetti con Icarus Verilog ( `iverilog` ), Verilator o ModelSim/Questa:

```
`timescale 1ns/1ns

module tb_mio_modulo;

    parameter WIDTH = 8;

    logic             clk;
    logic             reset;
    logic             en;
    logic [WIDTH-1:0] d;
    logic [WIDTH-1:0] q;

    // 1. Istanziazione del Device Under Test (DUT)
    register_en #(WIDTH) dut (
        .clk(clk),
        .reset(reset),
        .en(en),
        .d(d),
        .q(q)
    );

    // 2. Generatore di Clock (periodo 10ns → 100MHz)
    always #5 clk = ~clk;

    // 3. Vettore di Stimoli
    initial begin
        // Salvataggio tracce per visualizzatore GTKWave
        $dumpfile("waves.vcd");
        $dumpvars(0, tb_mio_modulo);

        // Inizializzazione segnali
        clk = 0;
        reset = 1;
        en = 0;
        d = '0;

        // Rilascio del reset dopo 20ns
        #20 reset = 0;

        // Test 1: Caricamento con enable
        #10;
        en = 1;
        d = 8'hA5;

        #10;
        // Verifica con Asserzione
        assert (q === 8'hA5)
            $display("[PASS] Test 1 superato con successo!");
        else
            $error("[FAIL] Errore! Valore atteso 0xA5, trovato %h", q);

        // Test 2: Enable basso (il dato deve restare 0xA5)
        #10;
        en = 0;
        d = 8'h3C;

        #10;
        assert (q === 8'hA5)
            $display("[PASS] Test 2 (hold) superato!");
        else
            $error("[FAIL] Il registro non ha mantenuto il valore!");

        #20;
        $display("Tutti i test completati.");
        $finish;
    end

endmodule
```

---

# PARTE 2: VHDL (RIGORE, LIBRERIE E STRUTTURA)

## 1. ARCHITETTURA DEL CODICE VHDL

In VHDL la descrizione è rigorosamente divisa in due parti: 1. `entity`: Specifica l'interfaccia verso l'esterno (porte e generics). 2. `architecture`: Specifica il comportamento interno o la struttura circuitale.

### 1.1 Librerie Indispensabili (Template Fisso d'Esame)

All'inizio di ogni file VHDL inserire sempre:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;     -- Definisce STD_LOGIC e STD_LOGIC_VECTOR
use IEEE.NUMERIC_STD.ALL;        -- Standard per operazioni matematiche e conversioni
```

### 1.2 I 9 Valori di `STD_LOGIC`

Mentre in C un booleano vale 0 o 1, in VHDL `std_logic` può valere: - `'0'`: Zero logico (forzato) - `'1'`: Uno logico (forzato) - `'Z'`: Alta impedenza (disconnesso / tri-state) - `'X'`: Conflitto o valore sconosciuto (es. due porte che guidano 0 e 1 contemporaneamente sullo stesso filo) - `'U'`: Non inizializzato (Uninitialized - tipico valore visto nei registri all'avvio della simulazione se manca il reset!)

---

## 2. I TIPI DI ASSEGNAMENTO IN VHDL: SEGNALI VS VARIABILI

### 2.1 Segnali (`signal`, con operatore `<=`)

- **Dove:** Dichiarati nell'`architecture` (tra `architecture` e `begin`).
- **Semantica:** Rappresentano **fili fisici**.
- **Comportamento temporale:** L'assegnamento `<=` pianifica l'aggiornamento per il ciclo successivo ("delta delay" o fronte di clock). Non cambia immediatamente il suo valore durante l'esecuzione del blocco corrente.

### 2.2 Variabili (`variable`, con operatore `:=`)

- **Dove:** Dichiarate **SOLO** all'interno dei blocchi `process` (tra `process` e `begin`).
- **Semantica:** Aggiornamento **immediato**, esattamente come le variabili in C.
- **Scopo:** Utili per contatori di cicli di loop `for`, calcoli complessi a passi intermedi o variabili locali algoritmiche.
- **Attenzione:** Le variabili non esistono fuori dal loro `process`.

### 2.3 Tabella di Confronto Critica

| Proprietà | Segnale ( `signal` ) | Variabile ( `variable` ) |
|---|---|---|
| **Operatore** | `<=` | `:=` |
| **Dichiarazione** | Nell'Architecture | Solo dentro il Process |
| **Aggiornamento** | Differito (Delta cycle o Clock) | Immediato |
| **Hardware Reale** | Filo fisico o Registro Flip-Flop | Filo temporaneo / logica locale |

---

## 3. ASSEGNAMENTI CONCURRENT (COMBINATORI FUORI DAI PROCESS)

In VHDL fuori dai `process` tutto è concorrente. Esistono 3 modi principali per scrivere logica:

### 3.1 Assegnamento Diretto e Porte Logiche

```
y_not  <= not a;
y_and  <= a and b;
y_or   <= a or b;
y_xor  <= a xor b;
y_nand <= a nand b;
y_nor  <= a nor b;
y_xnor <= a xnor b;
```

### 3.2 Assegnamento Condizionale ( `when - else` ) -> MUX 2:1

```
y <= d1 when sel = '1' else d0;
```

### 3.3 Assegnamento Selettivo ( `with - select` ) -> MUX 4:1

```
with sel select
   y <= d0 when "00",
        d1 when "01",
        d2 when "10",
        d3 when "11",
        (others => '0') when others;
```

---

## 4. LOGICA SEQUENZIALE IN VHDL ( `process` )

### 4.1 La Sensitivity List e il Fronte di Clock

Il blocco per la logica sequenziale è il `process`. Per rilevare il fronte di salita si usa la funzione moderna **`rising_edge(clk)`** (preferibile alla vecchia dizione `clk'event and clk = '1'`).

### Flip-Flop con Reset Asincrono

```
process(clk, reset)
begin
    if reset = '1' then
        q ≤ (others ⇒ '0');
    elsif rising_edge(clk) then
        q ≤ d;
    end if;
end process;
```

### Registro con Reset ed Enable

```
process(clk, reset)
begin
    if reset = '1' then
        q ≤ (others ⇒ '0');
    elsif rising_edge(clk) then
        if en = '1' then
            q ≤ d;
        end if;
    end if;
end process;
```

---

## 5. FINITE STATE MACHINES (FSM) IN VHDL

Schema a 2 processi standard:

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fsm_moore is
    port (
        clk   : in  std_logic;
        reset : in  std_logic;
        a     : in  std_logic;
        y     : out std_logic
    );
end entity;

architecture Behavioral of fsm_moore is

    -- 1. Definizione degli stati simbolici
    type state_type is (S0, S1, S2);
    signal current_state, next_state : state_type;

begin

    -- PROCESSO 1: Registro di Stato (Sincrono)
    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= S0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

    -- PROCESSO 2: Logica di Prossimo Stato (Combinatorio)
    process(current_state, a)
    begin
        -- Valore di default
        next_state <= current_state;

        case current_state is
            when S0 =>
                if a = '1' then
                    next_state <= S1;
                else
                    next_state <= S0;
                end if;

            when S1 =>
                if a = '0' then
                    next_state <= S2;
                else
                    next_state <= S1;
                end if;

            when S2 =>
                if a = '1' then
                    next_state <= S1;
                else
                    next_state <= S0;
                end if;

            when others =>
                next_state <= S0;
        end case;
    end process;

    -- USCITA (Moore: dipende solo dallo stato corrente)
    y <= '1' when current_state = S2 else '0';

end architecture;
```

## 6. MODELLAZIONE STRUTTURALE (COMPONENT E PORT MAP)

In VHDL, per collegare più moduli insieme creando una gerarchia (come visto nel circuito MAC):

```vhdl
architecture Structural of top_system is

    -- 1. Dichiarazione del componente da istanziare
    component full_adder is
        port (
            a, b, cin : in  std_logic;
            s, cout   : out std_logic
        );
    end component;

    -- Segnali interni di interconnessione
    signal c_intermedio : std_logic;

begin

    -- 2. Istanziazione con associazione esplicita delle porte
    fa0: full_adder port map (
        a    => in_a(0),
        b    => in_b(0),
        cin  => '0',
        s    => out_s(0),
        cout => c_intermedio
    );

    fa1: full_adder port map (
        a    => in_a(1),
        b    => in_b(1),
        cin  => c_intermedio,
        s    => out_s(1),
        cout => out_cout
    );

end architecture;
```

---

# PARTE 3: CONFRONTO DIRETTO E CHECKLIST D'ESAME

## 1. TABELLA DI CONVERSIONE RAPIDA SYSTEMVERILOG $\Longleftrightarrow$ VHDL

| Concetto Hardware | SystemVerilog | VHDL |
|---|---|---|
| **Porta / Interfaccia** | `input logic [7:0] a` | `a : in std_logic_vector(7 downto 0);` |
| **Segnale interno** | `logic [7:0] s;` | `signal s : std_logic_vector(7 downto 0);` |
| **Costante / Parametro** | `parameter N = 8` | `generic (N : integer := 8);` |
| **Tutti zeri su bus** | `'0` | `(others => '0')` |
| **Concatenazione** | `{a, b}` | `a & b` |
| **Replicazione** | `{4{a}}` | `(3 downto 0 => a)` |
| **NOT / AND / OR / XOR** | `~`, `&`, `\|`, `^` | `not`, `and`, `or`, `xor` |
| **XNOR** | `~^` | `xnor` |
| **Assegnamento Combinatorio** | `assign y = a & b;` | `y <= a and b;` |
| **MUX 2:1 Condizionale** | `assign y = s ? d1 : d0;` | `y <= d1 when s='1' else d0;` |
| **MUX Con Scelta** | `case (sel) ... endcase` | `with sel select ...` |
| **Blocco Combinatorio** | `always_comb begin ... end` | `process(all) begin ... end` |
| **Blocco Sequenziale FF** | `always_ff @(posedge clk)` | `process(clk) if rising_edge(clk) ...` |
| **Assegnamento Sequenziale** | Non-blocking `<=` | Segnale `<=` |
| **Variabile Locale Immediata** | Bloccante `=` | Variabile `:=` |
| **Stati FSM** | `typedef enum logic [1:0] {S0, S1} state_t;` | `type state_t is (S0, S1);` |
| **Alta Impedenza** | `'bz` | `'Z'` |
| **Testbench Delay** | `#10;` | `wait for 10 ns;` |

---

## 2. CHECKLIST DI VERIFICA PRIMA DI CONSEGNARE L'ESERCIZIO

Prima di considerare concluso un esercizio d'esame, esegui mentalmente questa lista di controllo:

- [ ] **Ho usato il tipo di assegnamento corretto?**
- In SV: Tutti gli `always_ff` usano solo `<=`, tutti gli `always_comb` usano solo `=`.
- In VHDL: Non ho confuso segnali ( `<=` ) e variabili ( `:=` ).
- [ ] **Ho evitato latch involontari?**
- Tutti i blocchi combinatori ( `always_comb` o `process` ) hanno un'assegnazione di default in cima, oppure tutti i rami `if` hanno il relativo `else` e i `case` hanno il `default` / `others`.
- [ ] **Ho rispettato la regola del Singolo Driver?**
- Nessun segnale `logic` è pilotato da due parti diverse.
- [ ] **Reset appropriato:**
- Se l'esercizio richiede reset asincrono, è presente nella sensitivity list ( `@(posedge clk or posedge reset)` in SV; `process(clk, reset)` in VHDL).
- Se è sincrono, compare solo dentro l'`if(reset)` senza essere nella sensitivity list.
- [ ] **Dimensionamento dei bus (Bit-Width):**

- In somme e moltiplicazioni, ho considerato la crescita dei bit? (Un prodotto $N \times N$ produce $2N$ bit; una somma $N + N$ può produrre $N+1$ bit con il carry).
- [ ] **Aritmetica con segno:**
- Se i numeri possono essere negativi, ho dichiarato esplicitamente `signed` in SV o usato `signed` di `numeric_std` in VHDL?
- [ ] **FSM completa:**
- Tutti gli stati gestiscono tutte le combinazioni degli ingressi.
- È sempre presente uno stato `default` che riporta allo stato iniziale di reset in caso di transizioni illegali.